IMPROVISED_REPO
State Reference

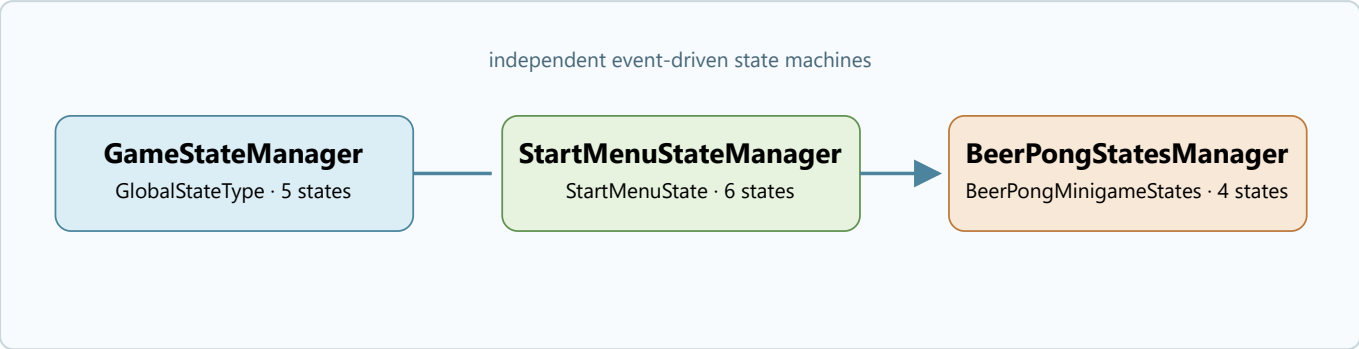
A code-grounded map of every state defined in the project, what each state controls, and how state changes are requested.

Generated 16 October 2026 · Unity C# source analysis

> **Scope.** The project contains three `IGameState`-based managers (global, start menu, and Beer Pong) plus an auxiliary camera/view enum inside `MiniGameSwitch`. The auxiliary enum is documented separately because it is not managed by `IGameState`.

State systems at a glance

Each manager owns state instances, subscribes to an event, calls `ExitState()` on the old state, selects the target, then calls `EnterState()`. The manager calls `UpdateState()` every frame.

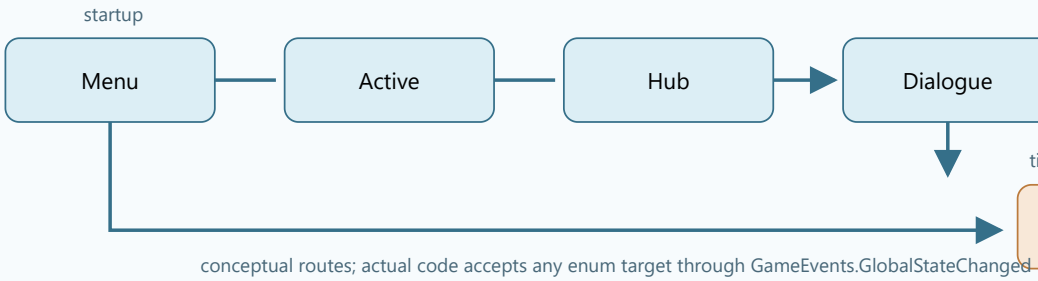


Documented managed states

- **Global:** Menu, Active, Hub, Dialogue, Outcome.
- **Start menu:** Start, TaskHandout, LevelSelect, LevelChoose, Settings, Await.
- **Beer Pong:** PlayerTurn, AITurn, PlayerWin, AIWin.
- **Auxiliary view:** MiniGame, MainGame, LeftGame, RightGame.

> **Implementation status.** Several states are structural placeholders: their `EnterState`, `UpdateState`, and `ExitState` methods are currently empty. Those states are still listed because they are real enum values and are instantiated by a manager.

Global state flow



The manager starts in `Menu`. `PlayerInputManager` toggles Menu/Active with Escape and ends dialogue with Escape. `DialogueManager` requests Dialogue when a dialogue starts and restores the prior global state when it ends. `GameManager` and `BombManager` request Outcome for loss conditions; `GameManager` explicitly handles timer expiry.

GLOBAL · GlobalStateType.Menu

Menu



Enter

Writes a debug log indicating entry.

Update

No per-frame behavior.

Exit

Writes a debug log indicating exit.

Controls

Represents menu/input mode at the global layer; Escape changes it to `Active`.

Source: GameStateManager.cs, MENU.cs, PlayerInputManager.cs

GLOBAL · GlobalStateType.Active

Active



Enter

Writes a debug log indicating entry.

Update

No per-frame behavior.

Exit

Writes a debug log indicating exit.

Controls

Represents active gameplay/input mode; Escape changes it to `Menu`.

Source: GameStateManager.cs, ACTIVE.cs, PlayerInputManager.cs

GLOBAL · GlobalStateType.Hub

Hub



Enter

Writes a debug log indicating entry.

Update

No per-frame behavior.

Exit

Writes a debug log indicating exit.

Controls

No state-specific behavior is currently implemented. It is available as a global state target.

Source: GameStateManager.cs, HUB.cs

GLOBAL · GlobalStateType.Dialogue

Dialogue



Enter

Only logs entry.

Update

No per-frame behavior.

Exit

Only logs exit.

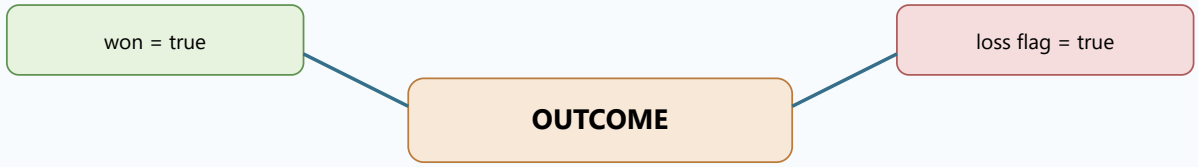
Controls

`PlayerInputManager` sends Enter/E to advance dialogue and Escape to request its end. `DialogueManager` tracks the state before dialogue and restores it on exit.

Source: GameStateManager.cs, DIALOGUE.cs, PlayerInputManager.cs, DialogueManager.cs

GLOBAL · GlobalStateType.Outcome

Outcome



Enter

Shows the win screen when `GameSessionData.won` is true. Otherwise shows the game-over screen when `lostOnTime`, `lostOnStrikes`, or `lost` is true.

Update / Exit

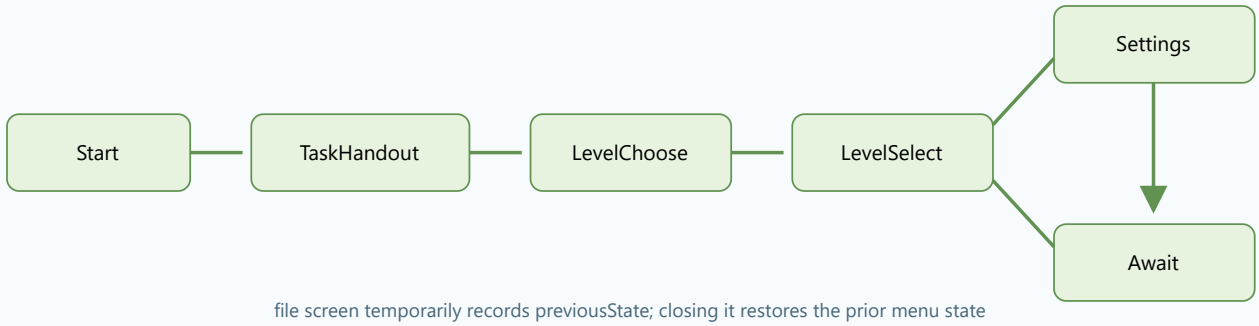
Update is intentionally empty. Exit hides both game-over and win screens, ensuring stale outcome UI is cleared.

Controls

Reached by timer expiry and other level/bomb loss or completion paths that request `GlobalStateType.Outcome`.

Source: OUTCOME.cs, GameManager.cs, GameSessionData.cs, BombManager.cs

Start-menu state flow



file screen temporarily records previousState; closing it restores the prior menu state

Primary requests: Start button → TaskHandout; dialogue button → LevelChoose after folder spawn; level button → LevelSelect; settings button toggles Settings/Await; drawer button toggles LevelChoose/Await. The file screen closes through `RestorePreviousStartMenuState()`.

START MENU · StartMenuState.Start

Start



Enter

Moves the camera immediately to marker 0.

Update

No per-frame behavior.

Exit

Fades out the start button canvas group over the default tween duration.

Controls

Initial state on `StartMenuStateManager.Start()`.

Source: StartMenuStateManager.cs, START.cs, StartButton.cs

START MENU · StartMenuState.TaskHandout

TaskHandout



Enter

Requests the NPC interaction sequence to enter. The start-screen manager moves the camera to marker 1, walks the NPC to the player, then shows dialogue UI.

Update / Exit

Update is empty. Exit requests the NPC interaction sequence to exit; the NPC walks away.

Controls

Entered by `StartButton.PressStart()`.

Source: TASKHANDOUT.cs, StartButton.cs, StartScreenManager.cs

START MENU · StartMenuState.LevelChoose

LevelChoose



Enter

Requests the drawer to open.

Update

No per-frame behavior.

Exit

Requests the drawer to close.

Controls

Entered by the dialogue button after the assignment folder has spawned, or by the drawer toggle.

Source: LEVELCHOOSE.cs, DialougeButton.cs, DrawerExitButton.cs

START MENU · StartMenuState.LevelSelect

LevelSelect



Enter / Update / Exit

All three lifecycle methods are currently empty. The state is a marker for the open level file screen.

Controls

A level button first changes to LevelSelect and then requests file-screen content. The manager stores the previous state so closing the file screen can restore it. If the previous state is LevelSelect, restore falls back to Await to avoid reopening the same file-screen state.

Source: LEVELSELECT.CS, LevelButton.cs, StartMenuStateManager.cs, StartScreenManager.cs

START MENU · StartMenuState.Settings

Settings



Enter

Requests settings menu open; the start-screen manager looks the camera at marker 4 with FOV 16.

Update

No per-frame behavior.

Exit

Requests settings menu close; the camera looks back at marker 0.

Controls

The settings button toggles based on whether the new state is Settings.

Source: SETTINGS.cs, SettingsButton.cs, StartScreenManager.cs

START MENU · StartMenuState.Await

Await



Enter

Moves the camera to marker 0 over the default tween duration.

Update

No per-frame behavior.

Exit

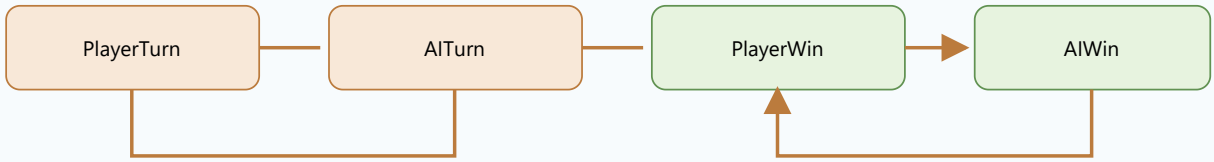
No behavior.

Controls

Used as the neutral state when closing settings or the drawer, and as the fallback after a level file screen closes.

Source: AWAIT.cs, SettingsButton.cs, DrawerExitButton.cs, StartMenuStateManager.cs

Beer Pong state flow

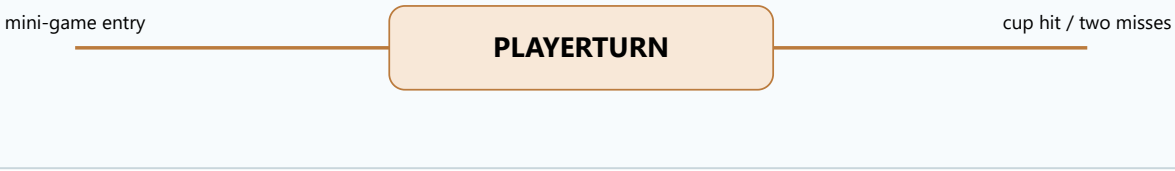


it / two misses alternate PlayerTurn ↔ AITurn; win enums are registered but not selected by BeerPong.cs in the inspected paths

The manager starts in `PlayerTurn`. `BeerPong.HandleCupWin` and `HandleMiss` alternate player and AI turns after a reset coroutine. The view switch also explicitly requests `PlayerTurn` when entering the mini-game.

BEER PONG · BeerPongMinigameStates.PlayerTurn

PlayerTurn



Enter / Update / Exit

The state class methods are currently empty. The manager nevertheless reports this as the current state and invokes its lifecycle.

Controls

Player interaction is enabled by Beer Pong logic when `CurrentState == PlayerTurn`. A successful hit or two misses advance to `AITurn`.

Source: BeerPongStatesManager.cs, PLAYERTURN.cs, BeerPong.cs, MiniGameSwitch.cs

BEER PONG · BeerPongMinigameStates.AITurn

AITurn



Enter / Update / Exit

The state class methods are currently empty. Beer Pong behavior is implemented in the owning `BeerPong` component.

Controls

AI-side hit/miss handling records AI cups and advances back to `PlayerTurn` after a hit or two misses.

Source: BeerPongStatesManager.cs, AITURN.cs, BeerPong.cs

BEER PONG · BeerPongMinigameStates.PlayerWin

PlayerWin



Enter / Update / Exit

All lifecycle methods are currently empty.

Controls

Registered in `BeerPongStatesManager` and selectable through the event API, but no inspected Beer Pong path requests this state directly.

Source: BeerPongStatesManager.cs, PLAYERWIN.cs

BEER PONG · BeerPongMinigameStates.AIWin

AIWin

AIWIN

Enter / Update / Exit

All lifecycle methods are currently empty.

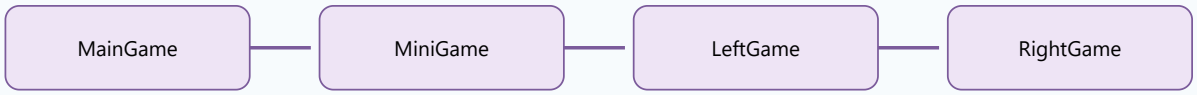
Controls

Registered in `BeerPongStatesManager` and selectable through the event API, but no inspected Beer Pong path requests this state directly.

Source: BeerPongStatesManager.cs, AIWIN.cs

Auxiliary MiniGameSwitch view states

These four values belong to the private `SwitchState` enum in `MiniGameSwitch`. They control camera/view placement and arrows; they do not implement `IGameState` and are independent of the Beer Pong turn manager.



Left/Right are camera positions; clicking the opposite arrow returns through MainGame

AUXILIARY VIEW · SwitchState.MiniGame

MiniGame

MINIGAME VIEW

Entry control

Sets FOV to 50, assigns MiniGame, requests Beer Pong `PlayerTurn`, moves the camera to the mini-game position, and hides level UI.

Exit / controls

Returning to MainGame shows level UI and moves to the main-game camera. Entry arrow visibility is updated to match the view.

Source: MiniGameSwitch.cs

AUXILIARY VIEW · SwitchState.MainGame

MainGame

MAIN GAME VIEW

Entry control

Moves the camera to the main-game position and shows level UI.

Controls

It is the initial value. Clicking left/right from the main view begins a coroutine to the corresponding side view; clicking the mini-game switch enters MiniGame.

Source: MiniGameSwitch.cs

AUXILIARY VIEW · SwitchState.LeftGame

LeftGame



Entry control

Starts a one-second camera interpolation to the left-game camera position and disables the left curved arrow while enabling the right navigation path.

Controls

Clicking the right arrow returns through `MainGame`; FOV is reset to 50 and arrow visibility is refreshed.

Source: MiniGameSwitch.cs

AUXILIARY VIEW · SwitchState.RightGame

RightGame

RIGHT GAME VIEW

Entry control

Starts a one-second camera interpolation to the right-game camera position and disables the right curved arrow while enabling the left navigation path.

Controls

Clicking the left arrow returns through `MainGame`; FOV is reset to 50 and arrow visibility is refreshed.

Source: MiniGameSwitch.cs

State-management notes

- Global and start-menu states are separate: changing one does not automatically change the other.
- All managed state managers use event subscriptions and unsubscribe in `OnDisable`.
- Unknown enum values are guarded in the start-menu and Beer Pong managers with a warning and a null current state. The global manager has no default case.
- State lifecycle methods are the intended extension points. Current gameplay behavior is often still implemented in managers/components that respond to the same events.
- `LevelSelect` is intentionally a lifecycle placeholder while the visible file-screen work is handled by `StartScreenManager`.

Prepared from the state managers, enum definitions, state classes, and direct event call sites in Assets/Scripts.